

parameters, *enough*, *parallel*, *warmup*, *repetitions*, and *cookie* are given with the default values.

## 5. Benchmarks

`lmbench` contains a large number of micro-benchmarks that measure various aspects of hardware and operating system performance. The benchmarks generally measure latency or bandwidth, but some new benchmarks also measure instruction-level parallelism.

Table 2 contains the full list of micro-benchmarks in `lmbench3`. Benchmarks that were converted to measure performance under scalable load are shown in *italics*, while the remaining benchmarks are shown with normal typeface. A detailed description of most benchmarks can be found in [12].

## 6. Scaling Benchmarks

There are a number of issues associated with converting single-process benchmarks with a single process to scalable benchmarks with several independent processes, in addition to the various issues addressed by the timing harness. Many of the benchmarks consume or utilize system resources, such as memory or network bandwidth, and a careful assessment of the likely resource contention issues is necessary to ensure that the benchmarks measure important aspects of system performance and not artifacts of artificial resource contention.

For example, the Linux 2.2 and 2.4 kernels use a single lock to control access to the kernel data structures for a file. This means that multiple processes accessing that file will have their operations serialized by that lock. If one is interested in how well a system can handle multiple independent accesses to separate files and if the child processes all access the same file, then this file sharing is an artificial source of contention with potentially dramatic effects on the benchmark results.

### 6.1. File System

A number of the benchmarks measure aspects of file system performance, such as `bw_file_rd`, `bw_mmap_rd`, `lat_mmap`, and `lat_pagefault`. It is not immediately apparent how these benchmarks should be extended to the parallel domain. For example, it may be important to know how file system performance scales when multiple processes are reading the same file, or when multiple processes are reading different files. The first case might be important for large, distributed scientific calculations, while the second might be more important for a web server.

However, for the operating system, the two cases are significantly different. When multiple processes access the same file, access to the kernel data structures for that file must be coordinated and so contention and locking of those structures can impact performance, while this is less true when multiple

| Name                    | Measures                                                                                                                                                                        |
|-------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Bandwidth</b>        |                                                                                                                                                                                 |
| <i>bw_file_rd</i>       | read and then load into processor                                                                                                                                               |
| <i>bw_mem</i>           | read, write, and copy data to/from memory                                                                                                                                       |
| <i>bw_mmap_rd</i>       | read from <code>mmap</code> 'ed memory                                                                                                                                          |
| <i>bw_pipe</i>          | <code>pipe</code> inter-process data copy                                                                                                                                       |
| <i>bw_tcp</i>           | TCP inter-process data copy                                                                                                                                                     |
| <i>bw_unix</i>          | UNIX inter-process                                                                                                                                                              |
| <b>Latency</b>          |                                                                                                                                                                                 |
| <i>lat_connect</i>      | TCP connection                                                                                                                                                                  |
| <i>lat_ctx</i>          | context switch via <code>pipe</code> -based "hot-potato" token passing                                                                                                          |
| <i>lat_dram_page</i>    | DRAM page open                                                                                                                                                                  |
| <i>lat_fcntl</i>        | <code>fcntl</code> file locking "hot-potato" token passing                                                                                                                      |
| <i>lat_fifo</i>         | FIFO "hot-potato" token passing                                                                                                                                                 |
| <i>lat_fs</i>           | file creation and deletion                                                                                                                                                      |
| <i>lat_http</i>         | http GET request latency                                                                                                                                                        |
| <i>lat_mem_rd</i>       | memory read                                                                                                                                                                     |
| <i>lat_mmap</i>         | <code>mmap</code> operation                                                                                                                                                     |
| <i>lat_ops</i>          | basic operations ( <i>xor</i> , <i>add</i> , <i>mul</i> , <i>div</i> , <i>mod</i> ) on (relevant) basic data types ( <i>int</i> , <i>int64</i> , <i>float</i> , <i>double</i> ) |
| <i>lat_pagefault</i>    | page fault handler                                                                                                                                                              |
| <i>lat_pipe</i>         | <code>pipe</code> "hot-potato" token passing                                                                                                                                    |
| <i>lat_pmake</i>        | time to complete <i>N</i> parallel jobs that do <i>usecs</i> -worth of work                                                                                                     |
| <i>lat_proc</i>         | procedure call overhead and process creation using <code>fork</code> , <code>fork</code> and <code>execve</code> , and <code>fork</code> and <code>/bin/sh</code>               |
| <i>lat_rpc</i>          | SUN RPC procedure call                                                                                                                                                          |
| <i>lat_select</i>       | <code>select</code> operation                                                                                                                                                   |
| <i>lat_sem</i>          | semaphore "hot-potato" token passing                                                                                                                                            |
| <i>lat_sig</i>          | signal handle installation and handling                                                                                                                                         |
| <i>lat_syscall</i>      | <code>open</code> , <code>close</code> , <code>getppid</code> , <code>write</code> , <code>stat</code> , <code>fstat</code>                                                     |
| <i>lat_tcp</i>          | TCP "hot-potato" token passing                                                                                                                                                  |
| <i>lat_udp</i>          | UDP "hot-potato" token passing                                                                                                                                                  |
| <i>lat_unix</i>         | UNIX "hot-potato" token passing                                                                                                                                                 |
| <i>lat_unix_connect</i> | UNIX socket connection                                                                                                                                                          |
| <i>lat_usleep</i>       | <code>usleep</code> , <code>select</code> , <code>pselect</code> , <code>nanosleep</code> , <code>setitimer</code> timer resolution                                             |
| <b>Other</b>            |                                                                                                                                                                                 |
| <i>disk</i>             | zone bandwidths and seek times                                                                                                                                                  |
| <i>line</i>             | cache line size                                                                                                                                                                 |
| <i>lmdd</i>             | <code>dd</code> clone                                                                                                                                                           |
| <i>par_mem</i>          | memory subsystem ILP                                                                                                                                                            |
| <i>par_ops</i>          | basic operation ILP                                                                                                                                                             |
| <i>stream</i>           | STREAM clones                                                                                                                                                                   |
| <i>tlb</i>              | TLB size                                                                                                                                                                        |

**Table 2. lmbench micro-benchmarks**

processes access different files.

In addition, there are any number of issues associated with ensuring that the benchmarks are either measuring operating system overhead (e.g., that no I/O is actually done to disk), or actually measuring the

system's I/O performance (e.g., that the data cannot be resident in the buffer cache). Especially with file system related benchmarks, it is very easy to develop benchmarks that compare apples and oranges (e.g., the benchmark includes the time to flush data to disk on one system, but only includes the time to flush a portion of data to disk on another system).

`lmbench3` allows the user to measure either case as controlled by a command-line switch. When measuring accesses to independent files, the benchmarks first create their own private copies of the file, one for each child process. Then each process accesses its private file. When measuring accesses to a single file, each child simply uses the designated file directly.

## 6.2. Context Switching

Measuring context switching accurately is a difficult task. `lmbench1` and `lmbench2` measured context switch times via a "hot-potato" approach using pipes connected in a ring. However, this experimental design heavily favors schedulers that do "hand-off" scheduling, since at most one process is active at a time. Consequently, it is not really a good benchmark for measuring scheduler overhead in multi-processor machines.

The design currently used in `lmbench3` is to create  $N$  `lmbench2`-style process rings and to measure the context switch times with all  $N$  rings running in parallel. This does extend the `lmbench2` context switch benchmark to a scalable form, but it still suffers from the same weaknesses.

One approach that was considered was to replace the ring with a star formation, so the master process would send tokens to each child and then wait for them all to be returned. This has the advantage that more than one process is active at a time, reducing the sensitivity to "hand-off" scheduling. However, this same feature can cause problems on a multi-processor system because several of the context switches and working set accesses can occur in parallel.

The design and methodology for measuring context switching and scheduler overhead need to be revisited so that it can more accurately measure performance for multi-processor machines.

## 7. Stream

`lmbench3` includes a new micro-benchmark, `stream` which measures the performance of John McCalpin's STREAM benchmark kernels for both STREAM version 1 [11] and version 2 [10]. This benchmark faithfully recreates each of the kernel operations from both STREAM benchmarks, and because of the powerful new timing harness it can easily measure memory system scalability.

Table 3 is based on McCalpin's tables [11][10] and shows the four kernels for each version of the `stream` benchmark. Note that the *read* columns

| Stream |                               |        |    |        |
|--------|-------------------------------|--------|----|--------|
| Kernel | Code                          | Bytes  |    | FL OPS |
|        |                               | rd     | wr |        |
| COPY   | $a[i] = b[i]$                 | 8(+8)  | 8  | 0      |
| SCALE  | $a[i] = q \times b[i]$        | 8(+8)  | 8  | 1      |
| ADD    | $a[i] = b[i] + c[i]$          | 16(+8) | 8  | 1      |
| TRIAD  | $a[i] = b[i] + q \times c[i]$ | 16(+8) | 8  | 2(-1)  |

| Stream2 |                               |       |    |        |
|---------|-------------------------------|-------|----|--------|
| Kernel  | Code                          | Bytes |    | FL OPS |
|         |                               | rd    | wr |        |
| FILL    | $a[i] = q$                    | 0(+8) | 8  | 0      |
| COPY    | $a[i] = b[i]$                 | 8(+8) | 8  | 0      |
| DAXPY   | $a[i] = a[i] + q \times b[i]$ | 16    | 8  | 2(-1)  |
| SUM     | $sum = sum + a[i]$            | 8     | 0  | 1      |

**Table 3. Stream operations**

include numbers in parentheses, which represent the average number of bytes read into the cache as a result of the write to that variable<sup>2</sup>. Cache lines are almost invariably bigger than a single double, and so when a write miss occurs the cache will read the line from memory and then modify the selected bytes. Sometimes vector instructions such as SSE and 3DNow can avoid this load by writing an entire cache line at once.

In addition, some architectures support multiply-add instructions which can do both the multiply and add operations for TRIAD and DAXPY in a single operation, so the physical FLOPS count would be 1 for these architectures on these instructions. The numbers in parenthesis in the *FLOPS* column reflect this reduction in FLOPS count.

Following the STREAM bandwidth reporting conventions, the `lmbench` STREAM benchmarks report their results as bandwidth results (MB/s) computed as a function of the amount of data explicitly read or written by the benchmark. For example, *copy* and *scale* copy data from one array to the other, so the bandwidth is measured as a function of the amount of data read plus the amount of data written, or the sum of the two array sizes. Similarly, *sum*, *triad*, and *daxpy* operate on three arrays, so the amount of data transferred is the sum of the sizes of the three arrays. Note that the actual amount of data that is transferred by the system may be larger because in the write path the cache may need to fetch (read) the cache line before a portion of it is overwritten by dirty data.

## 8. Unscalable benchmarks

There are a number of benchmarks which either did not make sense for scalable load, such as *mhz*, or which could not be extended to measure scalable load due to other constraints, such as *lat\_connect*.

<sup>2</sup> This number is independent of the cache line size because the STREAM uses dense arrays, so the cost is amortized over the subsequent operations on the rest of the line.

mhz measures the processor clock speed, which is not a scalable feature of the system, so it doesn't make any sense to create a version of it that measures scalable performance.

More specifically, `lat_connect` measures the latency of connecting to a TCP socket. TCP implementations have a timeout on sockets and there is generally a fixed size queue for sockets in the TIMEOUT state. This means that once the queue has been filled by a program connecting and closing sockets as fast as possible, then all new socket connections have to wait TIMEOUT seconds. Needless to say, this gives no insight into the latency of socket creation per se, but is rather a boring artifact. Since the `lmbench2` version of the benchmark can run for very short periods of time, it generally does not run into this problem and is able to correctly measure TCP connection latency.

Any scalable version of the benchmark needs each copy to run for at least a second, and there are  $N$  copies creating connections as fast as possible, so it would essentially be guaranteed to run into the TIMEOUT problem. Consequently, `lat_connect` was not enhanced to measure scalable performance.

## 9. Results

The results presented here were obtained using `lmbench` version 3.0-a2 under Linux 2.4.18-6mdk on a two processor 450MHz PIII running a stock Mandrake 8.2 Linux 2.4.18 kernel.

| Benchmark      | Latency ( $\mu$ s) |             |
|----------------|--------------------|-------------|
|                | 1 process          | 2 processes |
| null call      | 0.79               | 0.81        |
| null I/O       | 1.39               | 2.39        |
| stat           | 9.26               | 25.9        |
| open/close     | 11.7               | 27.1        |
| select (TCP)   | 55.3               | 58.6        |
| signal install | 1.89               | 1.95        |
| signal handler | 6.34               | 7.21        |
| fork process   | 793.               | 868.        |
| exec process   | 2474               | 2622        |
| sh process     | 24.K               | 25.K        |
| pipe           | 17.7               | 23.3        |
| unix socket    | 51.6               | 37.6        |
| UDP            | 70.2               | 70.6        |
| TCP            | 91.2               | 92.3        |
| rpc (UDP)      | 120.0              | 120.4       |
| rpc (TCP)      | 157.1              | 159.1       |

**Table 4. Latency results**

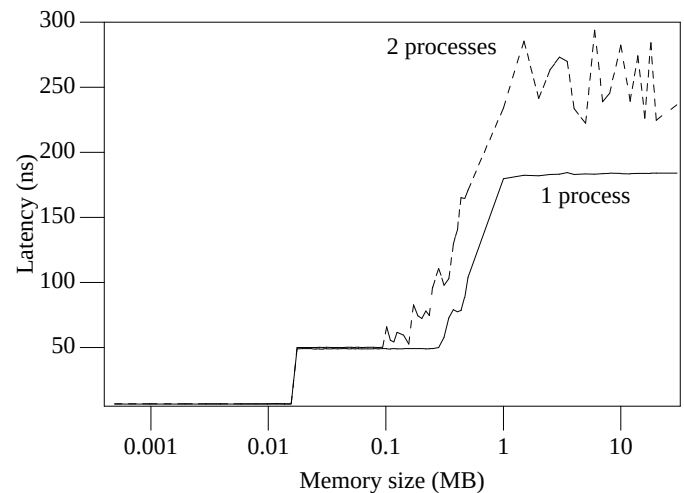
| Benchmark     | Load   |        |        |
|---------------|--------|--------|--------|
|               | 1      | 2      | 2clone |
| bw_file_rd    | 151.04 | 266.74 | 273.51 |
| bw_mmap_rd    | 316.08 | 480.02 | 482.57 |
| lat_mmap      | 615    | 878    | 786    |
| lat_pagefault | 2.9802 | 3.9159 | 3.4589 |

**Table 6. File bandwidth results**

| Benchmark     | Bandwidth (MB/s) |             |
|---------------|------------------|-------------|
|               | 1 process        | 2 processes |
| pipe          | 155              | 268         |
| unix socket   | 142              | 179         |
| TCP           | 57.5             | 57.8        |
| bcopy(libc)   | 134              | 175         |
| bcopy(hand)   | 144              | 174         |
| memory read   | 319              | 486         |
| memory write  | 199              | 202         |
| STREAM copy   | 288.68           | 367.99      |
| STREAM scale  | 290.39           | 369.08      |
| STREAM sum    | 337.75           | 415.54      |
| STREAM triad  | 246.90           | 380.09      |
| STREAM2 fill  | 198.96           | 276.28      |
| STREAM2 copy  | 288.55           | 359.93      |
| STREAM2 daxpy | 318.98           | 493.79      |
| STREAM2 sum   | 354.03           | 512.05      |

**Table 5. Bandwidth results**

Table 4 shows the latency of various system and communication operations for both 1 and 2 process loads, while Table 5 shows the bandwidth of various data operations and Table 6 shows how various file system operations scale. Table 6 shows system performance with one process, two processes sharing the same file, and two processes accessing their own files.



**Figure 4. Memory subsystem performance**

Figure 4 shows the memory latency curves with 32 byte strides for one and two process loads versus memory size.

## 10. Conclusion

`lmbench` is a useful, portable micro-benchmark suite designed to measure important aspects of system performance. `lmbench3` adds a number of important extensions, such as the ability to measure system